PROYECTO FINAL / MÉTRICAS Y ESTÁNDARES DE CALIDAD

Ice Cream App

Evaluación de mantenibilidad y evolución de una aplicación de pedidos

Campo	Información
Docente	José Miguel Llanos Mosquera
Equipo	Integrantes y aportes individuales por confirmar
Repositorio	github.com/pablo12968/ica.app
Fecha de la revisión	9 de noviembre de 2026
Estado	Borrador técnico para revisión del equipo

Introducción

Ice Cream App organiza la selección de helados y la confirmación de pedidos para recoger. El cliente consulta sabores, personaliza productos y revisa sus pedidos. La administración consulta cuentas, pedidos y eventos registrados por la API.

El objetivo del proyecto es vincular los requisitos con el código y sus pruebas, medir la mantenibilidad de los módulos y demostrar con una refactorización si se puede reducir la complejidad sin alterar los resultados verificados.

El alcance llega hasta el estado **Recibido** y la consulta de actividad. Pagos, reparto, selección de sucursal y seguimiento operativo quedan fuera del alcance. El modo demo ofrece un recorrido local con pedidos simulados.

Requisitos y trazabilidad

Cada fila identifica el resultado esperado, el componente que lo implementa y la evidencia disponible. Los casos Flutter se identificaron en el código y quedan pendientes de ejecución en esta revisión.

ID y función	Criterio de aceptación	Componente y evidencia
RF-01 Consultar catálogo	En modo conectado, el listado muestra solo productos disponibles con nombre y precio.	**ShopStore.loadProducts** y **product.routes.ts**. El caso Flutter de catálogo usa la demo. Consulta conectada pendiente.
RF-02 Buscar sabores	Al escribir parte del nombre, se muestran solo los sabores coincidentes.	**ShopHome.catalog**. Caso de búsqueda en widget_test.dart.
RF-03 Marcar favoritos	Al marcar un producto, aparece en la vista de favoritos. Al quitar la marca, deja de aparecer.	**ShopStore.toggleFavorite** y **ShopHome.catalog**. Marcado en widget_test.dart. Retiro por verificar.
RF-04 Personalizar helado	La línea conserva el tamaño y los toppings elegidos y suma sus suplementos al precio unitario.	**ProductPage**, **ToppingsPage** y **priceOrder**. Widget de compra y pruebas de extras del servidor.
RF-05 Editar cantidades	Al aumentar o disminuir unidades, el total cambia según el precio unitario. Una cantidad de cero retira la línea.	**ShopStore.changeQuantity**. Cálculo y eliminación en store_test.dart.
RF-06 Registrar cuenta	Con datos válidos se crea una cuenta de cliente. El mismo correo normalizado se rechaza con 409.	**AccountPage** y **POST /api/auth/register**. Integración en accounts.test.cjs.
RF-07 Iniciar sesión	Con credenciales válidas la API devuelve sesión. Una clave incorrecta devuelve 401.	**ShopStore.authenticate** y **POST /api/auth/login**. Integración en accounts.test.cjs.
RF-08 Confirmar pedido	Con sesión, carrito válido y total vigente, se crea un pedido con referencia y estado Recibido.	**CheckoutPage**, **ShopStore.checkout** y **POST /api/orders**. Integración y prueba del flujo demo.
RF-09 Ver historial propio	Después del pedido, el cliente puede consultar la orden en su historial. Una segunda cuenta no la recibe.	**ShopStore.refreshOrders** y **GET /api/orders**. Integración de aislamiento.
RF-10 Consultar actividad	El administrador puede ver el evento de un pedido confirmado en el listado paginado de actividad.	**AdminPage** y **GET /api/admin/activity**. Integración de eventos y permisos. Pantalla conectada pendiente.

Los RF con evidencia del servidor tienen respaldo en las pruebas de TypeScript compilado. La validación de pantalla en modo conectado requiere ejecutar Flutter frente a la API de prueba.

Requisitos no funcionales y arquitectura

ID y atributo	Criterio medible	Verificación y estado
RNF-01 Control de acceso	Pedidos sin sesión devuelven 401. Un cliente recibe 403 al consultar administración. Dos cuentas reciben historiales separados.	Integración en PostgreSQL. Se verificaron los tres controles.
RNF-02 Integridad	Un POST válido devuelve 201. El reintento con la misma clave y el mismo contenido devuelve 200 y el mismo ID. El historial conserva una sola orden.	Integración en PostgreSQL. Se verificaron los estados, la identidad de la orden y el tamaño del historial.
RNF-03 Mantenibilidad	Todas las unidades ejecutables de backend/src/order.ts deben tener V(G) ≤ 10.	Análisis sintáctico. Línea base con máximo 15. Copia refactorizada con máximo 9.

Arquitectura cliente servidor

El cliente organiza la presentación y el estado en Flutter. Express concentra el contrato HTTP, las reglas de validación y el control de acceso. PostgreSQL conserva los datos compartidos. Las pantallas pueden evolucionar con un contrato HTTP estable y la API puede probarse con una base aislada.



Figura 1. Dependencias principales entre los componentes de Ice Cream App.

La API recalcula el importe con el catálogo vigente y registra el pedido y su actividad dentro de una transacción. Flutter estima el total para informar al cliente. Los suplementos se definen en ambos lados, lo que exige mantenerlos sincronizados. La estructura combina Dart y TypeScript. El equipo debe confirmar con el docente que acepta esta combinación dado el requisito de lenguaje de la guía.

Ejecución y verificación

Ruta	Responsabilidad
frontend/lib	Interfaz de catálogo, personalización, carrito, cuenta y administración. ShopStore gestiona estado, preferencias y solicitudes HTTP.
backend/src	API Express, autenticación, validación y cálculo del pedido. config/database.ts configura el acceso a PostgreSQL.
backend/schema.sql	Catálogo de ejemplo y tablas de usuarios, sesiones, pedidos y actividad.
frontend/test y backend/test	Pruebas de interfaz, estado, cuentas, pedidos y reglas de servidor.

Instalación y demostración

Se requiere Flutter 3.47.2 con Dart 3.13.2, Node.js 22 o superior y PostgreSQL para el modo conectado. En una instalación nueva, copia **backend/.env.example** a **.env** dentro de backend y configura una base de desarrollo. Los comandos se ejecutan por bloque, en los directorios indicados.

```
backend> npm install
backend> npm run db:setup
backend> npm start

frontend> flutter pub get
frontend> flutter run -d chrome --web-port 8080 --dart-define=API_URL=http://localhost:3000
```

Para el modo local de demostración, inicia Flutter con **--dart-define=DEMO_MODE=true**. El modo conectado en navegador requiere que el origen coincida con **CORS_ORIGIN**. El README amplía la ejecución en Android y la habilitación del perfil de administración.

Evidencia de pruebas

Servidor. Se compiló TypeScript con Node.js 24.20.0. Pasaron ocho pruebas con PostgreSQL 17.11 aislado y **TEST_DATABASE=1**. Se incluyeron precio manipulado, extras, contraseñas, aislamiento, reintento, auditoría y control de acceso. Se usó **--test-isolation=none** por una restricción de creación de subprocesos del entorno.

Cliente. Se identificaron pruebas en widget_test.dart, store_test.dart, accounts_test.dart y sounds_test.dart. Flutter se detuvo antes de ejecutar la suite por restricciones del entorno. La ejecución de **flutter test** y la demostración conectada quedan pendientes.

Fichas de métricas

La revisión cubre el código de aplicación en **backend/src** y **frontend/lib**. El alcance excluye pruebas, SQL, activos, dependencias y archivos de plataforma. Los resultados incluyen la huella SHA-256 de cada archivo para identificar el estado local medido.

Métrica	Cálculo y umbral	Justificación y acción
V(G) Complejidad ciclomática	V(G) = 1 + decisiones por unidad ejecutable. Se incluyen bucles, condicionales, casos, capturas, ternarios y los operadores binarios &&, || y ??. Cada función anidada tiene su propio recuento. Bajo 1–10. Moderado 11–20. Alto 21–50. Muy alto >50. Referencia de la guía.	Identifica rutas con mayor esfuerzo de revisión y pruebas. Si V(G) supera 10, localizar las decisiones y separar validación, cálculo o presentación según corresponda.
LOC Líneas efectivas	Número de líneas que contienen al menos un token de código, incluida puntuación sintáctica. Los totales por archivo cuentan cada línea una vez. El rango de una unidad puede incluir unidades anidadas. Más de 50 LOC por unidad es una alerta de la guía.	Hace visible la concentración de código y el tamaño de los métodos de interfaz. Revisar si una unidad larga mezcla responsabilidades. Extraer widgets o funciones cuando haya un límite claro.
Dependencias internas directas Indicador de diseño	Cantidad de archivos internos distintos referidos por importaciones o reexportaciones estáticas reconocidas por archivo. Excluye dependencias externas y transitivas. Más de 3 destinos activa revisión. Umbral propuesto para ICA.	Señala archivos que coordinan muchos componentes. Revisar si las dependencias se explican por composición o si conviene dividir el módulo.

Herramientas y reproducción

Los medidores usan los árboles sintácticos de TypeScript 5.9.3 y Dart analyzer 10.1.0. Los scripts cuentan solo las construcciones indicadas, por lo que la complejidad es una aproximación sintáctica. El README de **metricas/** detalla la instalación y la salida.

```
node metricas/typescript.cjs "<ruta de ICA>"
cd metricas/dart
dart pub get
dart run medir.dart "<ruta de ICA>"
```

Resultados de la medición

Árbol	Archivos	LOC	Unidades	V(G) > 10	LOC > 50	Dep. > 3
TypeScript	11	286	40	3	0	1
Dart	8	3278	236	5	24	3
Total	19	3564	276	8	24	4

De 276 unidades, 268 quedaron en riesgo bajo, siete en riesgo moderado y una en riesgo alto. Ninguna superó 50 de complejidad. Los 24 intervalos con más de 50 LOC corresponden al cliente Flutter.

Archivos Dart con más líneas efectivas

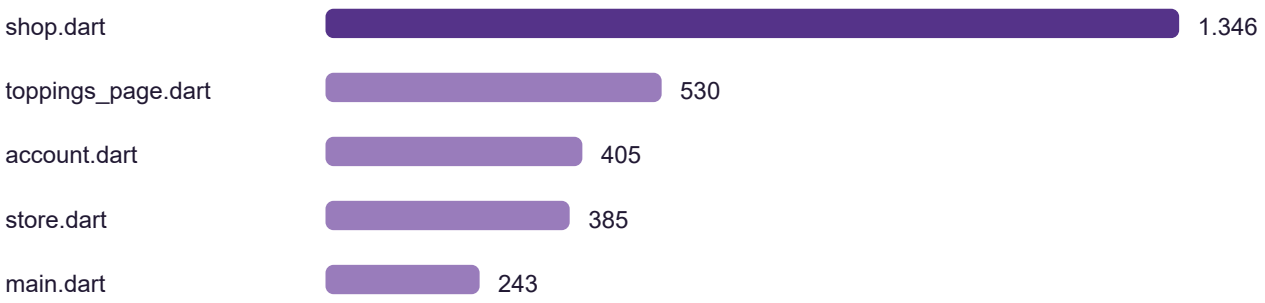


Figura 2. Líneas efectivas por archivo. shop.dart representa 1.346 de las 3.278 LOC del cliente.

Elemento	Resultado	Interpretación
auth.routes.ts · router.post · l. 8	V(G) **22** 37 LOC	Riesgo alto. El manejador comparte registro e inicio de sesión, con validación, persistencia y tratamiento de errores.
store.dart · checkout · l. 304	V(G) **16** 103 LOC	Riesgo moderado. El método gestiona los modos demo y conectado, los reintentos y el historial.
shop.dart · catalog · l. 236	V(G) **16** 189 LOC	Concentra búsqueda, filtros y construcción de la pantalla. La extracción de widgets puede reducir el tamaño de la unidad.
shop.dart · main.dart · app.ts · toppings_page.dart	Fan-out **7 / 6 / 6 / 4**	main.dart y app.ts ensamblan componentes. shop.dart combina pantallas y merece revisar sus dependencias junto con su tamaño.

Los resultados completos por archivo y unidad están en los JSON de docs/metricas/.

Refactorización y plan de trabajo

Se separaron la validación de contacto y la normalización de toppings en **backend/src/order.ts** mediante *Extract Method*. La ruta HTTP, el orden de las validaciones y el cálculo se conservaron en la copia. El parche se entrega por separado para su revisión e integración.

Indicador en order.ts	Antes	Después	Lectura
V(G) de validateOrder	15	8	Queda dentro de la meta de 10.
V(G) de la función de mapeo de ítems	15	9	La normalización pasa a su propia unidad.
V(G) máximo del archivo	15	9	Dos unidades excedían 10. Tras el cambio, ninguna.
LOC efectivas del archivo	26	32	Se añaden seis líneas por las declaraciones y llamadas.
Dependencias internas directas	0	0	La extracción queda en el mismo módulo.
Pruebas del servidor con PostgreSQL	8/8	8/8	Mismo conjunto de pruebas en ambas ejecuciones.

Las funciones nuevas obtuvieron V(G)=7 cada una. El máximo del proyecto sigue siendo 22 en autenticación. La reducción en pedidos es local y coincide con el alcance de RNF-03.

Priorización

Prioridad	Cambio propuesto	Riesgo, esfuerzo y efecto esperado
1 Autenticación	Separar la validación de credenciales y distinguir el flujo de registro del de inicio de sesión. Mantener transacciones y auditoría.	Riesgo alto por afectar sesión y cuentas. Esfuerzo medio. Reducir V(G) de 22 hacia 10 por unidad. Repetir integración y probar ambas rutas.
2 Confirmación	Extraer del método checkout la preparación de la solicitud y la persistencia del modo demo. Conservar el control de sesión y los reintentos.	Riesgo medio alto. Esfuerzo medio. Reducir V(G) de 16 y LOC por unidad. Ejecutar pruebas de historial, fallo de almacenamiento y reintento.
3 Catálogo	Extraer widgets de búsqueda y filtros y separar la composición del catálogo de sus reglas de filtrado.	Riesgo medio. Esfuerzo medio. Reducir las 189 LOC del método. Revisar el fan-out de shop.dart y el de los módulos resultantes.

Conclusiones y entrega

La separación cliente servidor permitió verificar con PostgreSQL los flujos de cuenta, pedido y administración. La revisión del cliente encontró mayor concentración de líneas en catálogo y pantallas. La prioridad del servidor es autenticación, donde el máximo de V(G) fue 22.

La refactorización de pedidos muestra el efecto de dividir responsabilidades. El máximo local bajó de 15 a 9 y se mantuvieron ocho pruebas aprobadas. El archivo ganó seis líneas efectivas. Evaluar las tres métricas en conjunto permite distinguir tamaño, complejidad y dependencias.

Aporte individual y control de versiones

El historial local registra **6d7882d**, incorporación del cliente y el servidor con autor Pablo Esteban Santana Vidal, y **4ecc8db**, commit inicial con autor pablo12968. La línea base corresponde al árbol local, que contiene cambios adicionales. Las huellas de los JSON identifican los archivos medidos. El equipo debe completar los roles, atribuir aportes con commits verificables e identificar la etiqueta de la versión final.

Pendientes antes de entregar

Frente	Comprobación pendiente
Alcance académico	Confirmar la aprobación del tema y la aceptación de Flutter/Dart junto con TypeScript. Completar integrantes y fecha oficial.
Demostración	Ejecutar las pruebas de Flutter y verificar el flujo conectado en el equipo de sustentación. La APK de demo anterior no incluye la integración actual de cuentas.
Repositorio	Revisar e integrar los cambios locales y el parche aprobado. Verificar el acceso del docente, los aportes individuales y la etiqueta de entrega.

Fuentes y declaración de IA

Llanos Mosquera, José Miguel. *Proyecto Final · Métricas y Estándares de Calidad*. Guía del curso, catálogo de métricas y rúbrica. Ice Cream App, código y pruebas del árbol local con base Git 6d7882d. Repositorio pablo12968/ica.app.

Para esta revisión se usó Codex como apoyo para inspeccionar el repositorio, preparar los medidores, ejecutar pruebas, proponer la refactorización y redactar la documentación. El README del repositorio registra el uso de ImageGen para recursos visuales. El equipo debe revisar el material y poder explicar las decisiones y el código en la sustentación.